

Análisis Arquitectónico — Sistema "Espagueti de Encuestas"

Fase 3 — Propuesta de Patrones de Diseño | Arquitectura de Software

 Escenarios	 Ilustraciones	 Principios SOLID	 Stacks
6	62	5	Spring Boot + Angular

Contexto del Documento

Anti-patrones detectados en Fases 1 y 2:

#	Anti-patrón	Descripción
1	God Controller	Controlador con todas las responsabilidades: HTTP, validación, SQL, BD
2	SQL Injection	SQL por concatenación de strings sin prepared statements
3	Tipado Débil	Map<String,Object> y any sin DTOs tipados
4	Hardcoded Config	Credenciales de BD en código fuente
5	Error Handling	try-catch con printStackTrace() y return null
6	Acoplamiento Frontend	HttpClient directo en componentes, URL hardcodeada

Escenario 1 — Layered Architecture (MVC)

Anti-patrón: God Controller | Patrón: Layered Architecture (MVC)

ANTES

```
@RestController
@CrossOrigin(origins = "*")
public class EncuestaController {

    private String URL = "jdbc:postgresql://db:5432/encuestas";
    private String USER = "espagueti";
    private String PASS = "espagueti";

    private JdbcTemplate jdbc() {
        DriverManagerDataSource ds = new DriverManagerDataSource();
        ds.setUrl(URL);
        ds.setUsername(USER);
        ds.setPassword(PASS);
        return new JdbcTemplate(ds);
    }
}
```

```

@PostMapping("/crear")
public Map crear(@RequestBody Map body) {
    String sql = "INSERT INTO encuestas(pregunta, si_count, no_count) VALUES
('"
        + body.get("pregunta") + "', 0, 0)";
    jdbc().update(sql);
    return body;
}
}

```

■ DESPUÉS

```

// Controller (solo HTTP)
@RestController
@CrossOrigin(origins = "*")
public class EncuestaController {
    private final EncuestaService encuestaService;

    @PostMapping("/crear")
    public ResponseEntity<EncuestaDTO> crear(
        @Valid @RequestBody CreateEncuestaRequest request) {
        return ResponseEntity.ok(encuestaService.crear(request));
    }
}

// Service (lógica de negocio)
@Service
public class SurveyService {
    private final SurveyRepository surveyRepository;

    public SurveyDTO create(CreateSurveyRequest request) {
        Survey survey = new Survey();
        survey.setPregunta(request.pregunta());
        survey.setSiCount(0);
        survey.setNoCount(0);
        return mapToDTO(surveyRepository.save(survey));
    }
}

```

Criterio	Antes	Después
Separación de responsabilidades	Nula — todo en Controller	Clara — 3 capas
Acoplamiento	Alto — Controller conoce BD	Bajo — interfaces
Testabilidad	Imposible aislar	Unit tests por capa

✗ SOLID Violados: SRP OCP DIP → ☒ **Aplicados:** SRP OCP DIP ISP **Madurez:** ● Bajo → ● Alto | **Calificación:** 2/10 → 9/10

🔍 Escenario 2 — Repository Pattern

Anti-patrón: SQL Concatenado | **Patrón:** Repository Pattern + Spring Data JPA

▣ ANTES

```
private JdbcTemplate jdbc() {
    DriverManagerDataSource ds = new DriverManagerDataSource();
    ds.setDriverClassName("org.postgresql.Driver");
    ds.setUrl(URL); ds.setUsername(USER); ds.setPassword(PASS);
    return new JdbcTemplate(ds);
}

String sql = "INSERT INTO encuestas(...) VALUES ('" + pregunta + "', 0, 0)";
jdbc().update(sql);

String sql = "UPDATE encuestas SET si_count = si_count + 1 WHERE id = " + id;
jdbc().update(sql);
```

▣ DESPUÉS

```
@Entity
@Table(name = "encuestas")
public class Encuesta {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String pregunta;
    private int siCount;
    private int noCount;
}

@Repository
public interface EncuestaRepository extends JpaRepository<Encuesta, Long> {}

// Uso en Service
Encuesta saved = encuestaRepository.save(encuesta);
Encuesta enc = encuestaRepository.findById(id)
    .orElseThrow(() -> new EncuestaNotFoundException());
```

✗ **Violados:** SRP OCP DIP LSP → ☒ **Aplicados:** SRP OCP DIP LSP ISP **Calificación:** 1.5/10 → **9.5/10**

🔍 Escenario 3 — DTO Pattern

Anti-patrón: Tipado Débil (Map) | **Patrón:** DTO Pattern + Jakarta Validation

▣ ANTES

```

@PostMapping("/crear")
public Map crear(@RequestBody Map body) {
    try {
        if (body == null || body.get("pregunta") == null
            || body.get("pregunta").toString().trim().length() < 3) {
            Map r = new HashMap();
            r.put("error", "Pregunta muy corta");
            return r;
        }
        String pregunta = body.get("pregunta").toString();
        String sql = "INSERT INTO encuestas(...) VALUES ('" + pregunta + "', 0,
0)";
        jdbc().update(sql);
        Integer id = jdbc().queryForObject("SELECT MAX(id) FROM encuestas",
Integer.class);
        Map resp = new HashMap();
        resp.put("id", id); resp.put("pregunta", pregunta);
        return resp;
    } catch (Exception e) { e.printStackTrace(); }
    return null;
}

```

DESPUÉS

```

public record CrearEncuestaRequest(
    @NotBlank(message = "La pregunta no puede estar vacía")
    @Size(min = 3, message = "Al menos 3 caracteres")
    String pregunta
) {}

public record EncuestaDTO(Long id, String pregunta, int siCount, int noCount) {}

@PostMapping("/crear")
public ResponseEntity<EncuestaDTO> crear(
    @Valid @RequestBody CrearEncuestaRequest request) {
    return ResponseEntity.ok(encuestaService.crear(request));
}

```

 **Violados:** SRP OCP ISP →  **Aplicados:** SRP OCP ISP DIP **Calificación:** 2/10 → 9/10

Escenario 4 — Externalized Configuration

Anti-patrón: Hardcoded Config | **Patrón:** Externalized Configuration

ANTES

```
private String URL = "jdbc:postgresql://db:5432/encuestas";
private String USER = "espagueti";
private String PASS = "espagueti";
```

■ DESPUÉS

```
# application.yml
spring:
  datasource:
    url: ${DB_URL}
    username: ${DB_USER}
    password: ${DB_PASS}
    driver-class-name: org.postgresql.Driver
```

```
# Docker environment
environment:
  - DB_URL=jdbc:postgresql://db:5432/encuestas
  - DB_USER=espagueti
  - DB_PASS=espagueti
```

Método `jdbc()` **ELIMINADO**. Spring Boot autoconfigura DataSource. ✖ **Violados:** SRP DIP OCP →

☑ **Aplicados:** SRP DIP OCP **Calificación:** 1/10 → 9.5/10

🔍 Escenario 5 — Global Exception Handler

Anti-patrón: Manejo Disperso | **Patrón:** @ControllerAdvice

■ ANTES

```
@GetMapping("/encuesta/{id}")
public Map encuesta(@PathVariable("id") Integer id) {
    try {
        if (id == null || id <= 0) { return null; }
        String sql = "SELECT id, pregunta, si_count, no_count "
            + "FROM encuestas WHERE id = " + id;
        return jdbc().queryForMap(sql);
    } catch (Exception e) {
        e.printStackTrace();
    }
    return null;
}
```

■ DESPUÉS

```

public class EncuestaNotFoundException extends RuntimeException {
    public EncuestaNotFoundException(Long id) {
        super("Encuesta con ID " + id + " no encontrada");
    }
}

public Encuesta buscarPorId(Long id) {
    return encuestaRepository.findById(id)
        .orElseThrow(() -> new EncuestaNotFoundException(id));
}

@ControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(EncuestaNotFoundException.class)
    public ResponseEntity<String> handleNotFound(EncuestaNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(ex.getMessage());
    }
    @ExceptionHandler(Exception.class)
    public ResponseEntity<String> handleGeneral(Exception ex) {
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body("Error interno del servidor");
    }
}

```

❌ Violados: SRP OCP DIP → ☒ Aplicados: SRP OCP DIP LSP Calificación: 1.5/10 → 9/10

🔍 Escenario 6 — Service Layer en Angular

Anti-patrón: HTTP en Componentes | **Patrón:** Service Layer + Environment Config

▣ ANTES

```

// crear.component.ts
constructor(private http: HttpClient, private router: Router) {}
url = 'http://localhost:8080';
this.http.post(this.url + '/crear', { pregunta: this.pregunta })

// encuesta.component.ts
constructor(private http: HttpClient, private route: ActivatedRoute) {}
this.http.get(this.url + '/encuesta/' + this.id)
this.http.post(this.url + '/votar', { id: this.id, voto: v })

// respuestas.component.ts
constructor(private http: HttpClient) {}
this.http.get(this.url + '/encuestas')
// URL hardcodeda repetida x3

```

▣ DESPUÉS

```
// environment.ts
export const environment = { apiUrl: 'http://localhost:8080' };

// encuesta.service.ts
@Injectable({ providedIn: 'root' })
export class EncuestaService {
  private api = environment.apiUrl;
  constructor(private http: HttpClient) {}

  crear(pregunta: string): Observable<any> {
    return this.http.post(`${this.api}/crear`, { pregunta });
  }
  obtenerTodas(): Observable<any> {
    return this.http.get(`${this.api}/encuestas`);
  }
  obtenerPorId(id: number): Observable<any> {
    return this.http.get(`${this.api}/encuesta/${id}`);
  }
  votar(id: number, voto: string): Observable<any> {
    return this.http.post(`${this.api}/votar`, { id, voto });
  }
}

// crear.component.ts (refactorizado)
constructor(private encuestaService: EncuestaService) {}
this.encuestaService.crear(this.pregunta).subscribe(...)
```

❌ **Violados:** SRP DIP OCP → ☒ **Aplicados:** SRP DIP OCP ISP **Calificación:** 2/10 → 9/10

Tabla Resumen Global

Escenario	Anti-patrón	Patrón Aplicado	Antes	Después	Madurez
ESC-01	God Controller	Layered Architecture	● 2/10	● 9/10	Bajo → Alto
ESC-02	SQL Concatenado	Repository Pattern	● 1.5/10	● 9.5/10	Bajo → Alto
ESC-03	Tipado Débil	DTO Pattern	● 2/10	● 9/10	Bajo → Alto
ESC-04	Hardcoded Config	Externalized Config	● 1/10	● 9.5/10	Bajo → Alto
ESC-05	try-catch disperso	Global Exception Handler	● 1.5/10	● 9/10	Bajo → Alto
ESC-06	HTTP en Componentes	Service Layer Angular	● 2/10	● 9/10	Bajo → Alto

☒ Conclusión Arquitectónica Final

El sistema "**Espagueti de Encuestas**" partía de un estado arquitectónico crítico. La aplicación de **6 patrones** transforma el sistema de madurez **Bajo** a **Alto**.

Métrica	Valor
Promedio ANTES	1.7 / 10
Promedio DESPUÉS	9.2 / 10
Mejora Global	+441%
Principios SOLID	5/5 cumplidos
Madurez Final	Alto